Deep Learning

Notes

Author:
Arun S
Data Scientist
arunkumars.co.in.

Neural Networks

Neural Networks are a class of machine learning models inspired by the structure and functioning of the human brain, designed to learn complex, non-linear relationships between inputs and outputs. At their core, neural networks are function approximators that learn a mapping from input space to output space by composing multiple simple transformations.

This compositional structure allows them to model patterns that are impossible for linear models to capture.

A neural network does not rely on handcrafted rules or explicit feature engineering. Instead, it learns representations automatically from data, adjusting its internal parameters through optimization.

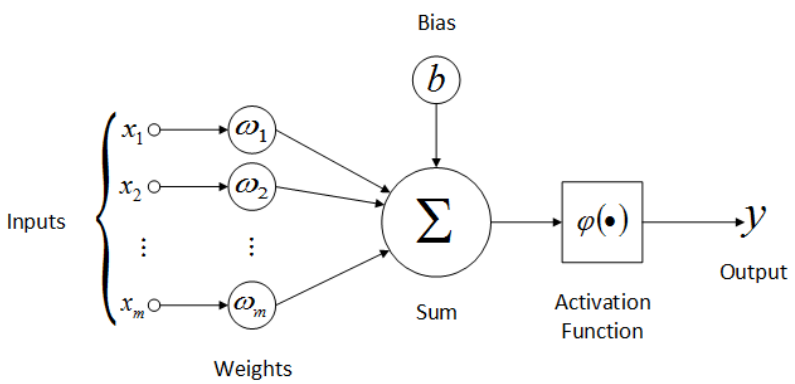
Role of Neural Networks in Modern AI

1. Neural networks underpin:
2. Computer vision systems
3. Speech recognition
4. Natural language processing
5. Recommendation systems
6. Large language models
7. Autonomous systems

Fundamental Building Block: The Neuron

The basic unit of a neural network is an artificial neuron. A neuron receives multiple inputs, applies a weighted sum, adds a bias term, and passes the result through a non-linear activation function.

This non-linearity is essential; without it, stacking multiple layers would collapse into a single linear transformation.



Layered Structure of Neural Networks

1. Neural networks are organized into layers. The input layer receives raw data.
2. One or more hidden layers transform the data through learned representations.
3. The output layer produces predictions such as class probabilities or numerical values.
4. Each hidden layer learns progressively more abstract features.
5. In image tasks, early layers may detect edges, intermediate layers may detect shapes, and deeper layers may recognize objects.
6. In language models, early layers capture syntax, while deeper layers encode semantics and reasoning patterns.

Forward Propagation

During forward propagation, data flows from the input layer through each hidden layer to the output. Each layer applies a linear transformation followed by a non-linear activation.

The output of one layer becomes the input to the next, creating a chain of transformations.

This forward pass defines a complex function whose shape is determined by millions or even billions of parameters. At initialization, this function is essentially random.

Learning consists of reshaping this function so that it aligns with the data.

Loss Functions and Learning Objective

The loss function converts model performance into a single scalar value that optimization algorithms can minimize.

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

The choice of loss function depends on the task. Classification problems typically use cross-entropy loss, while regression problems often use mean squared error.

Backpropagation

- Backpropagation is the algorithm that enables neural networks to learn.
- It computes how much each parameter contributed to the final loss by applying the chain rule of calculus backward through the network.
- Gradients are calculated layer by layer, starting from the output and moving toward the input.
- These gradients indicate the direction in which each parameter should be adjusted to reduce the loss.
- Without backpropagation, training deep networks would be computationally infeasible.

Optimization and Parameter Updates

Once gradients are computed, parameters are updated using an optimization algorithm such as gradient descent or its variants. Each update slightly modifies the network's function, improving performance on the training data.

This process of forward propagation, loss computation, backpropagation, and parameter update is repeated over many iterations until the network converges to a solution.

Generalization and Overfitting

A well-trained neural network should perform well not only on training data but also on unseen data. This ability is called generalization. Overfitting occurs when a network memorizes training data instead of learning underlying patterns.

Techniques such as regularization, dropout, data augmentation, and early stopping are used to encourage generalization.

Limitations of Neural Networks

Despite their power, neural networks have limitations. They require large amounts of data, are computationally expensive, can be difficult to interpret, and may fail unpredictably outside their training distribution

Activation Functions

Activation functions introduce non-linearity into neural networks, allowing them to model complex relationships beyond linear mappings. Without activation functions, no matter how many layers are stacked, a neural network would reduce to a single linear transformation and fail to capture real-world patterns.

At a conceptual level, an activation function determines whether a neuron should activate and by how much, shaping how information flows through the network.

Linear vs Non-Linear Activations

1. A purely linear activation does not add representational power.
2. When multiple linear layers are composed, they collapse into a single linear operation.
3. Non-linear activation functions break this collapse and enable depth to matter.

Sigmoid Activation Function

The Sigmoid function maps real-valued inputs into the range (0,1), making it historically useful for probabilistic interpretations.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid squashes large positive inputs toward 1 and large negative inputs toward 0.
While intuitive, this saturation causes **vanishing gradients**, making training deep networks difficult. As a result, sigmoid is rarely used in hidden layers today, though it remains common in output layers for binary classification.

Hyperbolic Tangent (tanh)

The tanh function is a scaled and shifted version of sigmoid, mapping inputs to the range (−1,1).

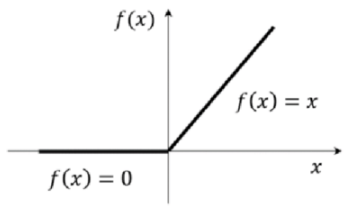
$$Tanh$$

$$f(x) = \frac{\left(e^{x} - e^{-x}\right)}{\left(e^{x} + e^{-x}\right)}$$

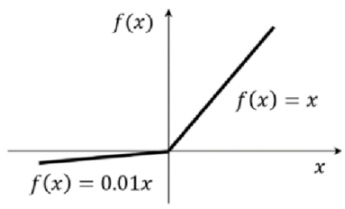
Because tanh is zero-centered, it improves optimization dynamics compared to sigmoid. However, it still suffers from saturation and vanishing gradients in deep networks.

Rectified Linear Unit (ReLU)

The **ReLU** activation is defined as the maximum of zero and the input.



ReLU activation function



LeakyReLU activation function

ReLU introduced a major breakthrough in deep learning by:

1. Avoiding saturation for positive inputs
2. Enabling sparse activations
3. Allowing efficient gradient propagation

Leaky ReLU

Leaky ReLU allows a small, non-zero gradient when the input is negative. This modification reduces the risk of neurons becoming inactive while preserving the simplicity of ReLU.

Soft max Activation

Soft max is used in output layers to convert raw scores into a probability distribution over classes.

Softmax

$$f_i(\vec{a}) = \frac{e^{a_i}}{\sum_k e^{a_k}}$$

Soft max ensures:

- All outputs are positive
- Outputs sum to one
- Probabilistic interpretation

The choice of activation function depends on:

1. Network depth
2. Task type
3. Optimization stability
4. Output interpretation

Gradient Descent

Gradient Descent is an **iterative optimization method** used to find parameter values that **minimize a loss function**.

In machine learning, training a model is equivalent to solving an optimization problem where the objective is to reduce prediction error. Since most loss functions in deep learning are **high-dimensional and non-convex**, closed-form solutions are impractical. Gradient Descent provides a scalable numerical solution.

At any point in parameter space, the gradient of the loss function indicates the direction of steepest increase. To minimize the loss, parameters are updated in the opposite direction of the gradient. Repeating this process gradually moves the parameters toward a local minimum.

Intuitively, gradient descent performs controlled downhill steps on the loss surface.

$$W_{new} = W_{old} - \alpha \underbrace{\frac{dJ}{dW}}_{\text{gradient}}$$

Role of the Learning Rate:

1. A small learning rate leads to stable but slow convergence
2. A large learning rate accelerates training but risks overshooting minima
3. Poor learning rate choice is the most common cause of training failure

Gradient Computation in Neural Networks

Gradients are computed using backpropagation, which applies the chain rule to efficiently propagate error signals from the output layer back through the network. Each parameter update depends on how much it contributed to the final loss.

Gradient descent and backpropagation together form the core learning mechanism of neural networks.

Types of Gradient Descent

Batch Gradient Descent:

Batch Gradient Descent computes the gradient of the loss function using **the entire training dataset** before updating the model parameters.

Each update reflects the true direction of steepest descent with respect to the full data distribution.

This method produces **deterministic and smooth updates**, since the same dataset is used every time.

Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent updates parameters using **a single randomly selected data point** at each iteration. Instead of following the exact gradient, it follows a **noisy estimate** of the gradient.

This noise introduces randomness into optimization, which can help escape shallow local minima and saddle points.

- Update rule for Stochastic Gradient Descent

$$\theta_j^i = \theta_j^i - \alpha(h_\theta(x^i) - y^i).\, x_j^i \;;$$

also written as;

$$\theta_j^i = \theta_j^i + \alpha(y^i - h_\theta(x)).\, x_j^i$$

Mini-Batch Gradient Descent

Mini-Batch Gradient Descent computes gradients using a **small subset of the dataset** (batch size typically between 16 and 1024). It combines the stability of batch gradient descent with the efficiency of SGD.

This is the **standard training method** used in deep learning.

Momentum Gradient Descent

Standard Gradient Descent updates parameters using only the **current gradient**, which can cause

1. Slow convergence in narrow valleys
2. Oscillations in directions with high curvature
3. Inefficient traversal of ravines

Momentum addresses this by accumulating past gradients, allowing the optimizer to build velocity in consistent directions and damp oscillations.

On iteration t:

Compute dW, db on the current mini-batch

$$v_{dW} = \beta v_{dW} + (1-\beta)dW$$

$$v_{db} = \beta v_{db} + (1-\beta)db$$

$$W = W - \alpha v_{dW},\;\; b = b - \alpha v_{db}$$

RMS Prop and Adaptive Learning Rates

RMS Prop adapts the learning rate **individually for each parameter** based on recent gradient magnitudes.

$$For\ each\ Parameter\ w^j$$

$$(j\ subscript\ dropped\ for\ clarity)$$

$$\nu_t = \rho\nu_{t-1} + (1-\rho) * g_t^2$$

$$\Delta\omega_t = -\frac{\eta}{\sqrt{\nu_t + \epsilon}} * g_t$$

$$\omega_{t+1} = \omega_t + \Delta\omega_t$$

$\eta : Initial\ Learning\ rate$
$\nu_t : Exponential\ Average\ of\ squares\ of\ gradients$
$g_t : Gradient\ at\ time\ t\ along\ \omega^j$

Adam Optimizer (Adaptive Moment Estimation)

Adam combines the strengths of Momentum and RMSProp

1. Momentum for direction
2. RMS Prop for adaptive step sizes

$$m_t = \beta_1 \cdot m_{t-1} + (1-\beta_1) \cdot g_t$$

$$v_t = \beta_2 \cdot v_{t-1} + (1-\beta_2) \cdot g_t^2$$

Optimizer	*Strength*	*Weakness*
Batch GD	Stable, exact	Poor generalization
SGD	Strong generalization	Noisy convergence
Momentum	Faster convergence	No adaptive scaling
RMS Prop	Stable updates	No momentum
Adam	Fast and adaptive	Sometimes worse generalization

1. Use Adam for fast prototyping and large models
2. Switch to SGD + Momentum when generalization matters
3. Mini-batch SGD is the industry standard
4. Optimizer choice affects where you converge, not just how fast

Vanishing and Exploding Gradients

Vanishing and exploding gradients are fundamental optimization problems that arise when training deep neural networks using gradient-based methods.

They describe situations where gradients propagated backward through the network become either excessively small or excessively large, preventing effective learning.

These issues directly affect how well information and error signals flow across layers and time steps.

Neural networks learn by applying **backpropagation**, which repeatedly applies the **chain rule of calculus** across layers. Each layer contributes a multiplicative factor to the gradient. As depth increases, this repeated multiplication can cause gradients to shrink toward zero or grow uncontrollably.

If derivatives are mostly smaller than 1, gradients vanish.
If they are mostly larger than 1, gradients explode.

Vanishing Gradients

Vanishing gradients occur when gradients decay exponentially as they are propagated backward through layers. When gradients become extremely small, early layers learn very slowly or stop learning entirely. This makes it difficult for deep networks to capture long-range dependencies or hierarchical features.

Activation functions like sigmoid and tanh have derivatives bounded by values less than 1. Repeated multiplication of these derivatives across layers causes gradients to shrink rapidly.

Exploding Gradients

Exploding gradients occur when gradients grow exponentially during backpropagation. Small changes in parameters lead to extremely large updates, causing unstable training and numerical overflow.

Exploding gradients are more common when weights are large or poorly initialized.

Vanishing vs Exploding Gradients (Contrast)

Vanishing gradients suppress learning by making updates too small.
Exploding gradients destabilize learning by making updates too large.

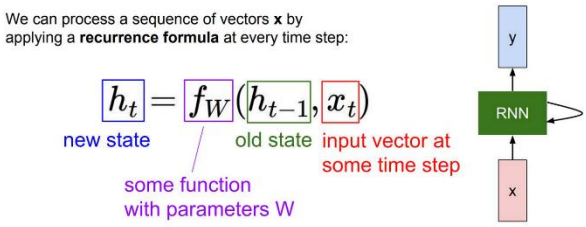
Both problems stem from **poor gradient flow**, not from data or labels.

Role of Activation Functions

- Sigmoid and tanh compress inputs and reduce gradient magnitude
- ReLU maintains gradients for positive inputs
- GELU and variants provide smooth yet stable gradients

Recurrent Neural Networks (RNN)

Recurrent Neural Networks (RNNs) are a class of neural networks designed to process sequential data, where the current output depends not only on the current input but also on previous inputs. Unlike feedforward networks, RNNs maintain an internal hidden state that acts as memory, allowing them to model temporal and sequential dependencies.



At each time step, the network:

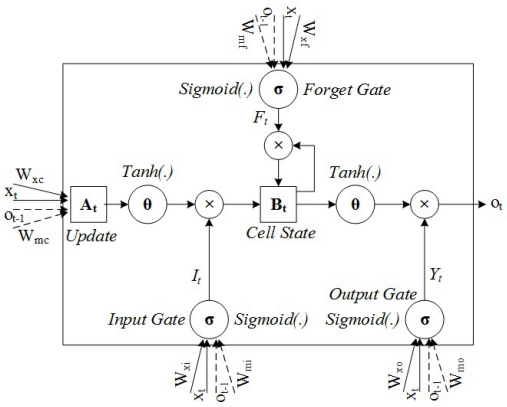
- Takes the current input
- Combines it with information from the previous time step
- Produces a new hidden state

Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) networks are a specialized type of Recurrent Neural Network (RNN) designed to overcome the vanishing gradient problem and to model long-term dependencies in sequential data.

LSTMs introduce a structured memory mechanism that allows the network to retain, update, or discard information selectively over long sequences.

Unlike vanilla RNNs, LSTMs explicitly control information flow using gates, making learning stable even across many time steps.



LSTM Gates and Inputs

Input - Provides the current time-step information to the LSTM, influencing both memory updates and output generation.

Previous Hidden State - Carries short-term contextual information from the previous time step and influences all gate decisions.

Previous Cell State - Stores long-term memory from earlier time steps and flows forward with minimal modification.

Forget Gate - Decides which parts of the previous cell state should be discarded or retained.

Input Gate - Controls how much new information should be written into the cell state.

Candidate Memory - Proposes new content that could be added to the cell state.

Cell State - Represents the updated long-term memory after forgetting and adding information.

Output Gate - Determines which parts of the cell state should influence the current output.

Hidden State- Acts as the exposed memory used for predictions and passed to the next time step.

Gated Recurrent Unit

A Gated Recurrent Unit (GRU) is a type of recurrent neural network designed to model sequential and temporal data while addressing the vanishing gradient problem of vanilla RNNs.

GRUs simplify the LSTM architecture by combining long-term and short-term memory into a single hidden state and using only two gates to regulate information flow. This simpler structure makes GRUs computationally efficient while still effective at capturing dependencies over time.

GRUs decide **how much past information to keep and how much new information to incorporate** at each time step.

GRU Gates and Inputs

Input - Provides the current time-step information that influences memory updates and output generation.

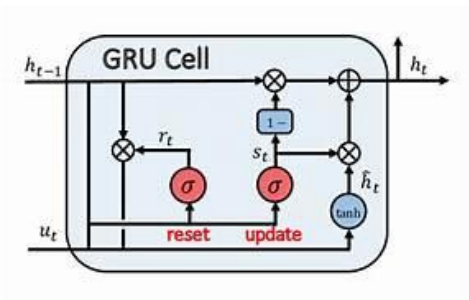
Previous Hidden State - Carries contextual information from the previous time step and serves as the sole memory in a GRU.

Update Gate - Controls how much of the previous hidden state should be retained versus updated with new information.

Reset Gate - Determines how much past information should be ignored when computing the candidate activation.

Candidate Hidden State - Proposes a new hidden state by combining the current input with selectively reset past information.

Hidden State - Represents both short-term and long-term memory and is passed forward as the model's state and output.



Convolutional Neural Networks (CNN)

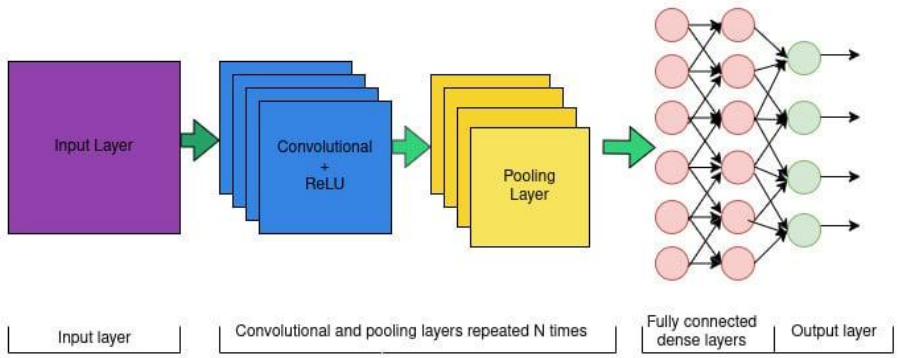
Convolutional Neural Networks (CNNs) are a class of deep learning models designed to process grid-structured data, most commonly images. Unlike fully connected networks, CNNs exploit the spatial structure of data by using local connections and shared weights.

This makes them highly efficient and effective for visual pattern recognition tasks such as image classification, detection, and segmentation.

The core strength of CNNs lies in their ability to learn hierarchical spatial features, progressing from simple patterns to complex structures.

CNNs are widely used in:

- Image classification
- Object detection
- Medical imaging
- Autonomous driving
- Video analysis



Core Building Blocks of CNNs

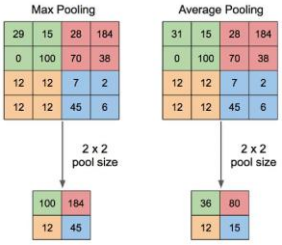
Convolution Operation - The convolution operation applies a small matrix called a filter (kernel) across the input image to produce a feature map. Each filter learns to detect a specific pattern such as edges, corners, or textures.

Stride, Padding, and Output Size - Stride controls how far the filter moves at each step. **Padding** adds extra pixels around the input to control spatial size. Together, they determine the resolution of the output feature map.

Feature Maps and Channels - Each convolutional filter produces one feature map. Using multiple filters allows the network to learn **multiple feature types in parallel**. As depth increases, feature maps capture more abstract patterns while spatial resolution typically decreases.

Activation Functions in CNNs - After convolution, a non-linear activation (commonly **ReLU**) is applied. This introduces non-linearity, enabling the network to model complex patterns. ReLU is favored for its simplicity and stable gradient behavior.

Pooling Layers - Pooling layers reduce spatial dimensions by aggregating information over local regions. The most common form, **max pooling**, keeps the strongest activation in each region.



Hierarchical Feature Learning

1. CNNs learn features hierarchically
2. Early layers detect edges and simple textures
3. Middle layers detect shapes and parts
4. Deep layers detect objects and semantics

Fully Connected Layers - In classification tasks, convolutional features are flattened and passed to **fully connected layers**, which combine learned features to produce final predictions. In modern CNNs, these layers are often minimized or replaced with **global average pooling** to reduce parameters

Common CNN Architectures:

1. Deep stacking of convolutions
2. Smaller kernels with greater depth
3. Skip connections for stability

Limitations of CNNs

1. Limited ability to model long-range dependencies
2. Sensitivity to rotation and viewpoint changes
3. Fixed receptive fields without attention